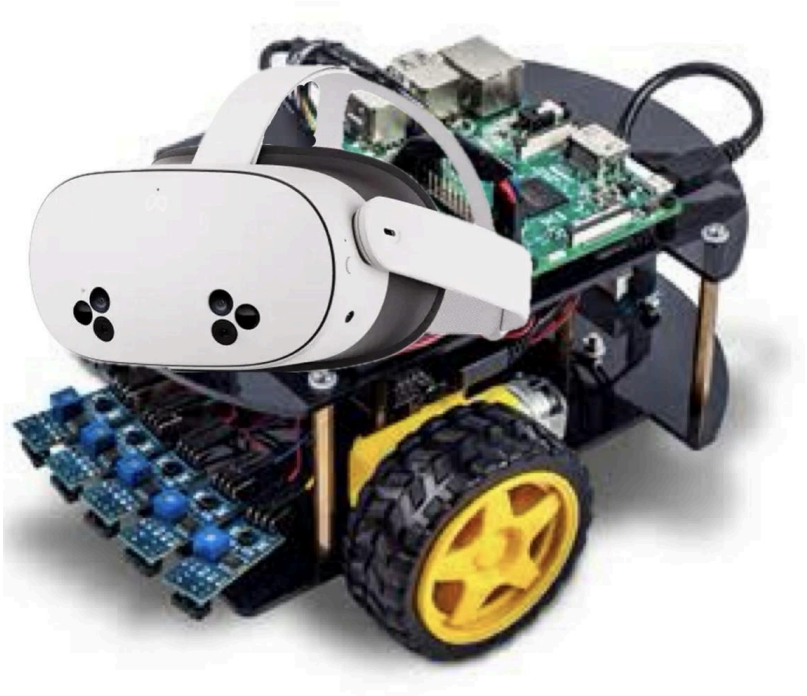
Virtual Robotics



Wesley Burcham, Matthew Palmer, Michael Brown

Table of Contents

- **Introduction (pg. 1-2)**
- **Executive Summary (pg. 2)**
- **Scenarios (pg. 2 - 3)**
- **Interface Transitions (pg. 3)**
- **Screen by Screen Breakdown (pg. 4)**
- **Errors and Edge Cases (pg. 5- 6)**
- **Technical Components (pg. 6)**
- **TODO (pg.7)**
- **Glossary (pg. 7 - 11)**

Introduction:

- **Summary of the problem our project solves:**

In robotics, there are many approaches to controlling a robot. Our clients, specifically college professors involved in robotics and mathematics, are looking for a control medium that allows them to maneuver the robot and observe sensor readings.

- **What our project is and how it solves the aforementioned problem:**
  The solution our team is implementing is a VR headset controller that can be used to control robot movement and observe sensor output. The main robot we'll be focusing on for this project is an OSOYOO Robot Pie Car equipped with a camera, a voltage meter, a tracking sensor, and an ultrasonic sensor. The headset we'll be using is the Meta Quest 3 equipped with 2 handheld joystick controllers and the actual headset. Our implementation will allow the user to control the robot via the joysticks and read from the camera and sensors using the headset.

Executive Summary:

The Virtual Robotics project is a WebXR Meta Quest application that will mainly aid Professor Hall's and Professor Benedict's research on real-world robotics applications. The main application functions are a live-feed robot camera and controller, as well as a menu to switch the inputs and outputs between multiple different robots to the headset (if a second robot is completed in time, which is outside the scope of this class' work).

The universality of the Pi robots will be a streamlined process that will only require small file modifications to be able to use a new robot that runs on a Raspberry Pi. After the completion of the project, all code and supplies used will be made public, allowing anybody to use, replicate, or expand upon our application.

The live camera footage of the robot camera, as well as the controls, will be used to efficiently control robots. It will consist of a start screen where you select any number of robots you will want to use from a menu selection screen highlighting them, and then a start button. Clicking the start button will then load the visuals of the first robot chosen, where you are shown the camera feed of the robot and a diagnostics see-through overlay that is on the side. This will have important information about the robot from the available sensors. You will also see a drop-down menu in which you will see an end session button to terminate the program along with settings to toggle the overlay on and off.

Scenarios:

In each of the following scenarios our actor can be a variety of different avatars. Our default is a researcher looking to expand the capabilities/use of VR in the context of robotics controls, but our user could just as easily be a recreationalist, a researcher of some other field (environmentalist, wildlife biologist, etc), each with the goal of using the platform for its mobility and observational capabilities.

- **Platform Testing for Different Kinds of Robots:**
  Users can adapt the platform to their robot of choice and have a means of testing different systems of that robot with a platform that allows them to display any sensor readings. For example, like in our case, in order to test the circuitry and the ability of the motors to move in coordination with one another, we wrote a script

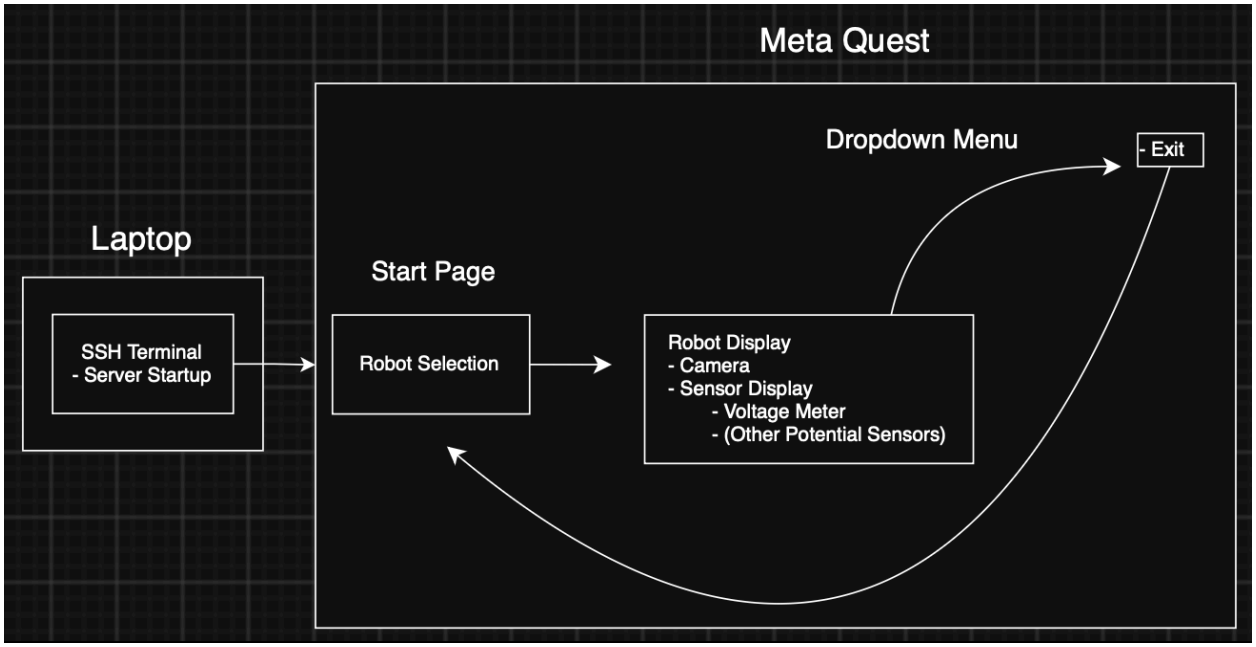
that allowed the Meta Quest's joysticks to communicate with the robot and tell it to move forward. The success of this test allowed us to validate that there was nothing wrong with the hardware and that the robot was able to receive signals, operate off of them, and respond back over a digital network.

- **Environment Exploration**
    - Users of the platform can access a camera feed of whatever sensor on their robot of choice they'd like, and depending on how small their robot of choice is, they can even use the platform to gain access to certain areas or crevices that they'd have a hard time getting into by themselves.
    - If the user adds different kinds of sensors to the robot and connects them to the display, they can also use the platform to see things that humans can't. Examples include thermal/infrared sensors, night vision cameras, and various sensors that are used to see particles in the air that are normally invisible/undetectable to human senses.

- **Recreation**

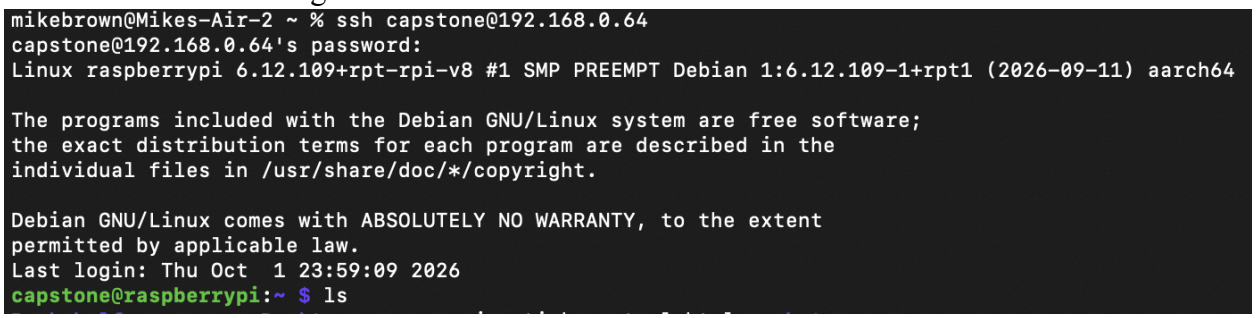
    The platform can also be used for entertainment/noneducational purposes, such as having a sensor's camera feed be used for video footage that can be uploaded onto the internet as either part of a funny YouTube video or as makeshift security footage that can be used to spy on insects or small animals (e.g. fruit flies, ants, small dogs/cats, etc.).

Interface Transitions

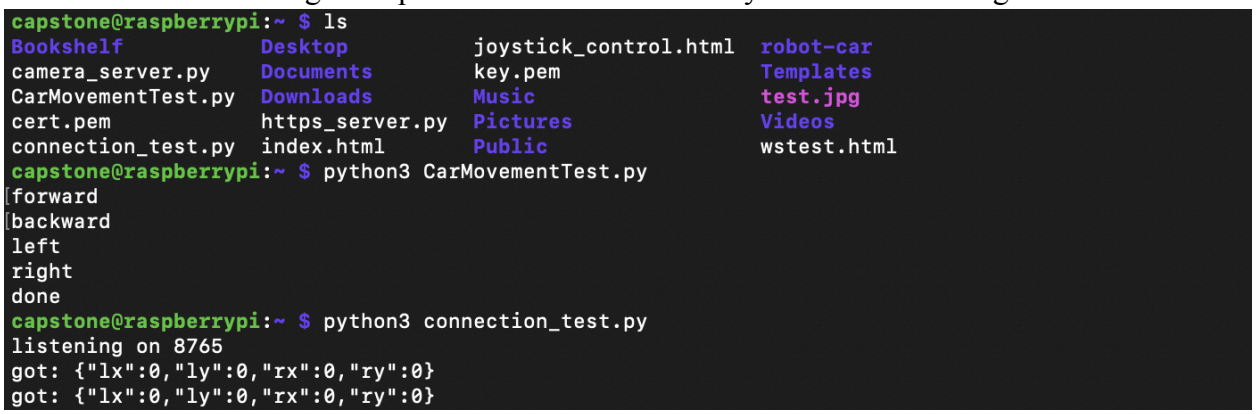


Screen by Screen Breakdown:

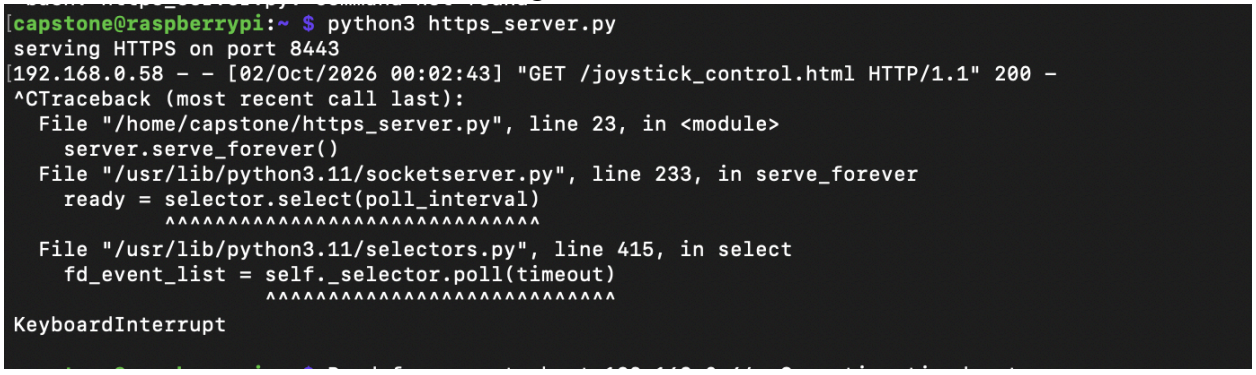
Terminal Screen showing SSH:



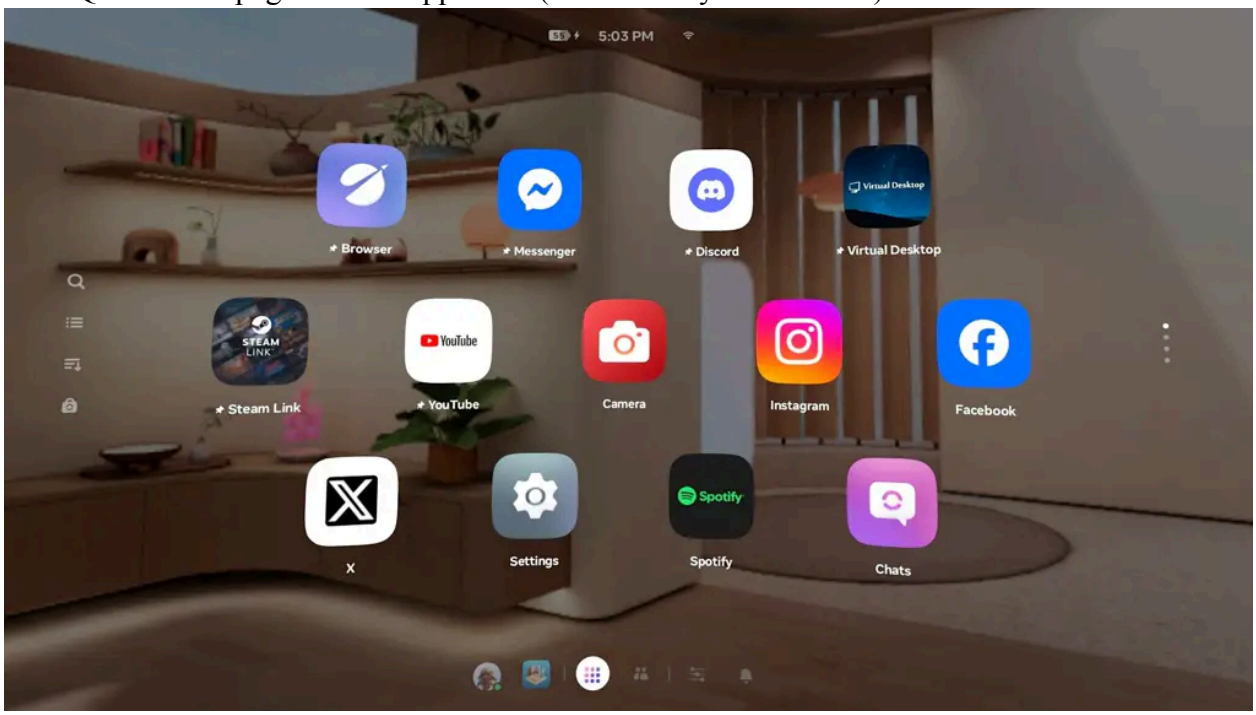
Terminal screen showing example of SSH terminal with Python socket hosting:



SSH Terminal with HTTPS server running:



Meta Quest home page with all app icons (most notably the browser):



Errors and Edge Cases:

This section describes the different things that may go wrong during the use of our product, the measures we took to mitigate or prevent them (if possible), and what the user should do if they come across them

Hardware:

If the user finds that they're having hardware issues (things won't turn on or run, motors turn awkwardly or are louder/quieter than usual, camera feed is extremely laggy (some lag is normal), etc.). Essentially, if there's an issue and the Raspberry Pi's name and IP address are visible on the network and the Pi can send and receive messages, the issue is likely with the hardware. As you'll find by reading below, most hardware solutions are solved easiest by replacing the component that is faulty or damaged. Even with the necessary expertise, fixing the components often requires more effort than it's worth. This does not cover extreme scenarios in which the user integrates more advanced hardware, in which case we are not qualified to suggest solutions. As for the solutions within our scope:

- **Dead/Faulty Batteries:**
  This can be diagnosed by observing that the robot is not turning on (no lights are visible on the computer boards or any of the on robot sensors). If the batteries are fully charged, the only way to isolate the issue is to use some sort of power meter to see if the battery's compartment is delivering power. The best solution is usually to just replace the batteries with newer ones to ensure that they aren't the issue. If the user is operating our instance of the project specifically, they will

have access to multiple spare batteries located in the RUNE laboratory with the permission of Dr. Robin Hall. We recommend fully charging the batteries before use as motor drivers consume a significant portion of battery power even when idle, giving the user about 1 hour of operation time with fully charged batteries.

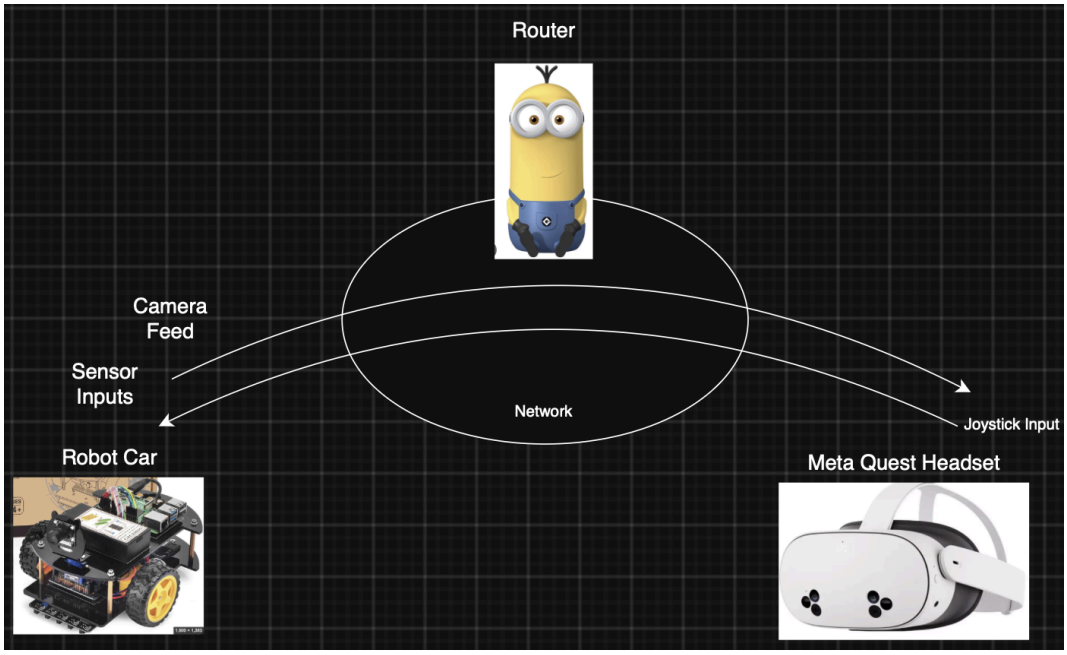
- **Dead Wires:**
  This can be diagnosed by observing that the robot is not turning on (no lights are visible on the computer boards or any of the on robot sensors) or if the robot is turning on but things like the motor or camera aren't receiving power. If the batteries are fully charged and functional and things still aren't receiving power this is the next likely issue assuming it isn't the component itself. One can use a power meter in a similar fashion to the solution above to verify the wire in question. There is no real technique for fixing this outside of replacing the wires themselves. Similar to the batteries, when using our platform, with the permission of Dr. Robin Hall or another Engineering faculty member, the user should be able to get access to wires and replace them.

- **Faulty Boards/Components:**
  - Similar to the others, this is best solved by replacing whatever component is causing the issue. If the system being controlled is more advanced than the one in this project, it is highly recommended that there be a means of testing subsets of the entire system such that if anything goes wrong, said tests can be run to isolate where the issue is coming from so that it can be addressed efficiently.

Digital/Networking Issues:
These issues usually come in the form of faulty network connections assuming it isn't just an error in the software being used in the case that the code from this project is altered. Assuming that is not the case, these issues can appear in a few ways, primarily with "choppy" controls and/or camera feed (repeatedly goes from working to not working, or simply just won't connect at all). To mitigate these issues, our team has implemented the use of a router to isolate the network and ensure that there is no traffic that would hinder our ability to connect the different parts of our system to each other. Any other issues would likely come from any software altered from what's found in this project. If it is, the best solution is to re-image the Raspberry Pi and re-upload the software.

Technical Components:

Network Diagram:
Fun Fact: In the electrical engineering community there is a router built like one of the minions from the Despicable Me series (Kevin). It's ironically one of the best routers on the market and our group wishes we found it before we started the project (we may end up getting it for final presentations).



TODO:

- How does the joystick input get sent?
- How does the joystick input get translated into motor inputs to move the actual car?
- How does the camera feed get back to the Meta Quest?

Glossary:

Microcontroller:
A microcontroller is a very small computer on an integrated circuit that can decide what processes happen and when on a variety of electronic devices, such as microwaves, thermostats and washing machines. Since it is very small, it's not as powerful as most of the bigger computers are as of this point in time due to having a small amount of memory and limited software capabilities, but on the brighter side of things, it doesn't use as much electricity as a bigger computer would, either.

Raspberry Pi:
A more powerful genre of microcontrollers(depending on what version you buy) that can do almost everything a regular computer can do while also not costing as much money to buy as that said computer. In this project, the Raspberry Pi will function as a translator between the VR headset and the motors.

Single Board Computer:
Essentially a miniature computer, similar to the ones people use almost every day, except that it is much less powerful (e.g. smaller amounts of memory, more limited software capabilities, etc) with the added benefit of not requiring as much electricity to use.

VR:
An acronym for Virtual Reality, which refers to the concept of an electronic device(typically a headset) using components such as cameras and sensors to run a program that overrides the real world camera feed and replaces it with a fully digitalized scene. In this project, this term will be used to refer to virtual reality headsets such as the Meta Quest and Apple Vision Pro.

AR:
Similar to Virtual Reality in concept, but this time, Augmented Reality(AR) only focuses on making digital objects appear as though they are actually in the real world via the aforementioned cameras and sensors.

Apple Vision Pro:
Apple's proprietary virtual reality platform(which may also double as a Mixed Reality (Virtual Reality(VR) + Augmented Reality(AR)) platform when used for activities outside the scope of this project. Compared to most other virtual reality headsets, all you need to do to control what the Vision Pro does is move your eyes and/or hands, or speak into the headset's microphone with your voice.

Meta Quest:
Meta's proprietary virtual reality platform that no longer requires a PC to run as of the Meta Quest 2's release. It was called the Oculus Quest about a decade ago, but had its name changed when Facebook's parent company changed its name to Meta in 2022 after buying the Oculus VR company on March 25, 2014. Compared to most other virtual reality headsets, you can download the Meta Horizon app on your smartphone to set it up instead of having to rely on a PC.

Router:
Manages data traffic between devices on a network. Will be used in this project to create an isolated network to minimize connection issues between the headset and the device.

IMU (Inertial Measurement Unit):
A small sensor chip that measures how a device is moving and tilting. It usually combines an accelerometer (measures changes in speed) and a gyroscope (measures rotation). Some also include a magnetometer, which works like a compass. In this project, it would report the robot's orientation.

WebXR Gamepad API:
A set of tools built into web browsers that lets a webpage read VR controller inputs, such as joystick position, button presses, and hand pose. This is how the app running in the Quest browser reads the controllers.

Hand Pose:
The position and orientation of the user's hand or controller in 3D space, as tracked by the headset.

JavaScript:
The programming language that runs inside web browsers. Here, it runs in the Quest browser to read the controllers and send commands.

Python Server:
A program written in Python that runs on the Raspberry Pi. It listens for incoming commands from the headset and translates them into motor instructions.

WebSocket:
A type of network connection that stays open, so data can flow in both directions continuously. It is well suited to real-time control because each joystick update doesn't need a brand-new connection.

WiFi (2.4GHz / 5GHz):
Wireless networking. The 5GHz band is faster and less crowded than 2.4GHz, which helps with low-delay video, but it has a shorter range and is worse at passing through walls.

Network Stack:
The layers of software in a device that handle sending and receiving data over a network.

Network Congestion:
Slowdowns that happen when too many devices or too much data are competing on the same network.

Isolated Network:
A private network, created here by the dedicated router, with only the project's devices on it. It avoids interference from other traffic.

Latency:
The delay between an action and its result, such as the time between moving the joystick and the robot responding, or between the camera capturing a frame and it appearing in the headset. Low latency is critical for comfortable VR control.

CSI Camera:
A camera that plugs directly into the Raspberry Pi's dedicated camera port through the Camera Serial Interface (CSI). It is faster and more efficient than a USB webcam because it connects straight to the Pi's image-processing hardware.

Ribbon Cable:
A flat, wide cable made of many thin wires side by side. It connects the CSI camera to the Raspberry Pi.

PWM (Pulse Width Modulation):
A way of controlling power by switching a signal on and off very quickly. The share of time the signal stays "on" (the pulse width) sets how much power gets through. A wider pulse makes a motor spin faster, and a narrower pulse makes it spin slower.

PWM HAT (PCA9685):
An add-on board that sits on top of the Raspberry Pi. "HAT" stands for Hardware Attached on Top. The PCA9685 chip generates steady, precise PWM signals on many channels at once, so the Pi doesn't have to produce the timing itself.

H-Bridge:
A circuit that can reverse the direction of the current flowing through a motor, which lets the motor spin both forward and backward. Motor driver boards are built around one or more H-bridges.

Motor Driver Board (Model X):
A board that takes the low-power control signals from the PWM HAT and switches the much larger battery current the motors need. The Pi can't power motors directly, so the driver board acts as the middleman.

Gear Motor:
An electric motor with a built-in gearbox. The gears trade speed for torque, which gives the motor enough turning force to move the robot's wheels.

I2C Bus:
A simple two-wire communication system that lets the Raspberry Pi talk to multiple chips, such as the PWM HAT or an IMU. Each device on the bus has its own address, so they can share the same wires.

ISP (Image Signal Processor):
A part of the Pi's chip that turns the raw data from the camera sensor into a usable image. It handles tasks like adjusting color, brightness, and noise.

Ethernet Cable:
A wired network cable. It is useful for setting up the Pi or router when a wireless connection isn't working yet.

H.264 Encoder:
Hardware or software that compresses video using the H.264 standard. It makes the video small enough to send over WiFi quickly. The Pi 4 has a dedicated chip that does this.

Decode:
Decompressing the encoded video on the receiving end so it can be displayed. Here, the Quest browser does the decoding.

Camera Feed:
The live video stream from the robot's camera.

Throttle:
The control that sets forward and backward speed. Here, it is the left joystick.

Yaw:
Rotation left or right around the vertical axis, meaning turning in place or steering. Here, it is controlled by the right joystick.

Orientation:
Which way the robot is facing and how it is tilted. It is usually described as yaw (turning), pitch (nose up or down), and roll (tilting side to side).

Project Management

GitHub Repository:
An online folder where the project's code is stored, versioned, and shared among team members.

Sprint:
A fixed block of time, often one to two weeks, focused on completing a specific set of goals for a project.